

Read the Artifact, Not the Flag: Honest Self-Evaluation and Recovery for Autonomous Robots

Gregory P. Braun
Forged Lucidity
Westbrook, Maine, USA
gpb4@cornell.edu

Abstract: A robot working with no person nearby must decide for itself whether its task is done, and a self-improving robot must also decide which of its own changes to keep. Can it make those calls honestly? Not with a grader learned from its own success reports: such a grader learns to pass the failures the robot cannot sense, and some that it can. In a simulated pick-and-place task on a model of the AgileX NERO arm, with a scripted policy, a classifier trained on the robot’s reports passed 1% of planted failures when every failure in its training data had been sensed and 76% when a cube could slip unnoticed from the gripper on 30% of carries, while its agreement with the robot’s own labels overstated its real accuracy. A fixed check that reads the final state from an overhead camera, in a scene built for it, passed none of the planted failures and matched ground truth on all 600 natural test episodes. The same check can drive repair: with a scripted routine acting only on the check’s image, the robot repaired 92% of its natural failures and every planted failure within reach, raising task success at the highest slip rate from 49% to 95%, and in 1,000 test episodes it never reported a failed task as done; when it could not repair a failure, it stopped and asked for help. When the robot tunes itself, its own settings can loosen its own check: a lower lift before the grip check raised its reported successes by 13 of 150 and cut true ones by 19. In a 50-round keep-or-kill loop (one run per arm), a grader retrained on the robot’s reports came to pass between 34 and 46 of 100 planted failures from round 10 on, while its reported success stayed within 4 points of the truth, short of the 10-point divergence we had registered; the same grader trained once on the outside check and frozen passed 1 of 100 throughout and let its loop undo a bad change. We distill three rules for evaluating robots that work alone (judge the final state, not the robot’s account of it; score from the logged artifact, not a status flag; fault-inject every grader before trusting it) and a protocol for measuring recovery that gives no credit for the robot’s own account. Next come hardware tests and a learned policy.

Keywords: autonomous robots, success detection, self-evaluation, failure recovery, fault injection

1 Introduction

A robot that works alone makes judgments no one checks at the time: whether its task is done, whether to try again, and, if it improves itself, which of its own changes to keep. Each judgment runs through a grader. In current systems that grader is often learned from the robot’s own experience: a success classifier trained on examples and rollouts [28, 20, 21], a vision-language model (VLM) that reads the final frame [7, 31], or a success detector frozen from an earlier checkpoint of the policy’s own model [10]. These graders work well enough to drive real improvement, and the papers that use them are candid that a policy can reach states that fool the classifier [28, 20]. In three of the systems we review, a person stays within reach of the grader’s mistakes [28, 3, 32]; a robot that works alone has no such person. This workshop asks what an honest benchmark looks like for agentic robotic

systems, and how to measure recovery without rewarding benchmark gaming. For a robot that works alone the question arrives before any benchmark, at the end of every task: can the robot tell, by itself, whether it is done?

Our answer is no if the grader shares the robot’s inputs, and yes if the grader reads the final state with an instrument the robot’s own reports never touched. A grader trained on the robot’s success labels inherits the robot’s blind spots: a failure the robot cannot sense enters the training data labelled as a success, and the grader learns to pass it. Agreement with the robot’s labels, the natural held-out check when no one is available to label, cannot reveal this, because the labels are wrong in the same place; planted failures can. We contribute:

- a controlled demonstration, on a simulated NERO arm with a scripted policy, that a success classifier trained on the robot’s own reports learns to pass unsensed failures although its input shows them, and that planted failures expose what held-out agreement hides (Section 4.1);
- evidence that a fixed outside check of the final state stays honest in this setting and can drive a scripted repair routine, the robot asking for help instead of claiming success when repair fails (Section 4.2);
- a closed 50-round self-improvement loop in which a grader retrained on the robot’s own reports decays where planted failures can see it and the success rate cannot, while a grader taught once by the outside check stays honest (Section 4.3);
- three rules for honest evaluation of robots that work alone, and a protocol for measuring recovery without rewarding gaming, whose resistance to a learning repair policy we argue but do not test (Section 3; next experiments in Section 6).¹

2 Related work

Learned success detectors. Singh et al. [28] learn a success classifier from examples and ask a person to label the states it calls successful, because the classifier is exploited through its false positives; SERL notes that the policy can reach states that fool its reward classifier [20], and HIL-SERL may collect extra data for the classifier’s false positives and negatives [21]. VLMs serve as success detectors [7] and zero-shot reward models [25], and both papers call for checks before such a model is trusted; optimizing hard against a learned proxy raises the same concern [9, 16]. A robot stack that asks an LLM or VLM to judge its own trace inherits the models’ limits as self-verifiers: they struggle to correct their own reasoning without external feedback [13], gain from sound external verifiers but not from self-critique [29], and favor their own generations [23].

Self-improving robot loops. In RoboCat, automated success detection drives hindsight relabelling while final results are counted by hand [3]; SOAR learns from autonomously collected data that a VLM labels [31]; Ghasemipour et al. [10] freeze a separate checkpoint for reward and success detection and report that pushing self-improvement past its peak can lower success; in ENPIRE, success detectors built by coding agents from minutes of human-labelled demonstrations supply both the reward and the measure of progress, with no reported error rate [30]. These systems keep the grader of record apart from the loop in some form; our rules require that separation and test it.

Recovery and evaluation practice. Inner Monologue feeds success detection back to a language-model planner [12], and REFLECT turns the robot’s multisensory record into a failure explanation that can guide a correction [19]; both need a signal that something went wrong. SIMPLER shows that simulated evaluation can track real-world performance once the control and visual gaps are reduced [18]; AutoEval deploys success classifiers only above 95% accuracy and checks its results against

¹The rules come from four failures of evaluation instruments in a logged, AI-assisted self-improvement process, recorded in Appendix A. Disclosure: that record and those four failures also underlie related work in preparation. The simulation studies, the three rules and the recovery protocol are this paper’s own and are not submitted elsewhere.

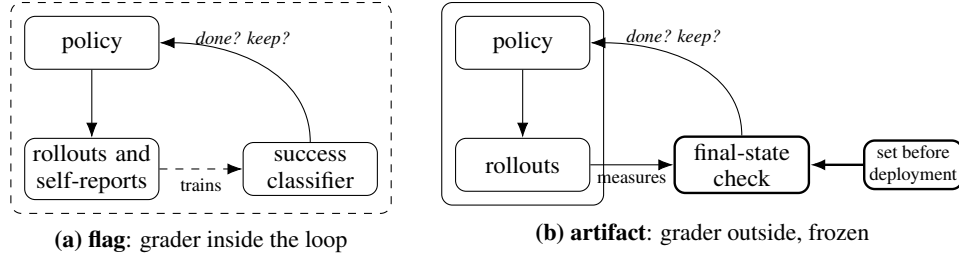


Figure 1: Two wirings of the grader that decides whether a task is done or a change is kept. (a) Trained on the policy’s own rollouts or self-reports, it shares the inputs it judges and reports a *flag*. (b) It reads the final state with an instrument the policy never trained against, fixed before deployment; once fault-injected (rule 3), a rise in its signal is a rise in the property.

evaluations done by hand [32]; Kress-Gazit et al. [17] ask papers to report the number of runs, the initial conditions and the success criteria. Scripted checks need the test too: in September 2026 Isaac Lab fixed a stacking check that “tested an instantaneous configuration, which a dropped cube satisfies on the way down” [14].

3 Three rules for honest self-evaluation

A robot that works alone grades itself after each task and, if it improves itself, at every keep-or-kill decision. The rules say how to wire that grading so that it stays honest while no one is watching.

1. **Judge the final state, not the robot’s account of it.** A robot’s account of what it did (a grasp check that read “held”, a task-complete flag, a language model’s “I verified X”) is output of the system being judged, not evidence that the property holds. Score the end state with a checker the robot did not write and was not trained against; where no physical end state exists, at least with a different instrument from the one graded.
2. **Read the artifact, not the flag.** Score from the logged record of what happened (an overhead image, a sensor trace, a log of files opened, a proof kernel’s verdict), never from the robot’s report of it or from a status flag that the reporting code sets.
3. **Fault-inject the grader before you trust it.** A checker that has not refused a planted fault has not been shown to check. Before a success classifier, a safety monitor or a benchmark reader is used to make a claim, feed it cases known to be failures and confirm, on the record, that it rejects them. This is the logic of mutation testing [5], which trusts a test suite only after it catches deliberately planted faults (here the grader plays the test suite and the planted failures play the mutants), and of test-oracle assessment, which scores an oracle by the faults it misses and the correct runs it rejects [15].

Working alone adds one constraint: the grader of record sits outside everything the robot’s own improvement may change. It is fixed and versioned before deployment and changed only when the robot reports back, on the record. A process that could rewrite its own evaluator would, under selection pressure, find the evaluator that passes it (specification gaming [16]; self-preference when the grader is a model [23]). A person’s role moves from watching each task to setting the check before the robot leaves and reading its log when it returns. That need not mean a hand-built check per task: a detector or VLM trained on labels that do not come from the robot can serve many tasks, if the robot cannot change it and its false-pass rate is measured, by planted failures, where it runs.

3.1 Measuring recovery without rewarding gaming

The workshop’s sixth open problem asks how to measure recovery without rewarding benchmark gaming; the rules give a protocol. (i) The harness, not the robot, plants each disturbance from a

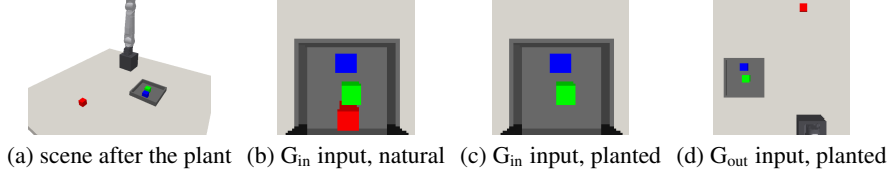


Figure 2: A development episode (seed 9003, $p_{\text{slip}} = 0.10$) after a planted failure. (a) The harness moved the red cube out; the arm is parked. (b, c) G_{in} 's 64 by 64 wrist image before and after the plant. (d) G_{out} 's overhead image. The self-report is PASS in both cases.

distribution fixed before the run, and logs it. (ii) A recovery counts only when the final state passes a check the robot does not control (ground truth in simulation, the outside grader on hardware); the robot's account of retrying earns nothing. (iii) Report three rates together: recoveries over failures; false completions, the share of episodes the robot reported as done whose final state fails; and requests for help. Self-grading hides the second, and a robot tuned only for the first can buy recoveries with false completions. (iv) Resetting the scene or hiding the object does not pay, because the check reads the task's final state. Section 4.2 applies the protocol.

4 The rules on a simulated NERO arm

We built both graders of Figure 1 around one scripted robot, let the robot repair its own failures with the artifact wiring alone, and then let it tune its own settings in a keep-or-kill loop, graded each way. No neural policy is trained; the loop tunes four settings of the scripted policy.

Setup. In PyBullet [4], a model of the AgileX NERO seven-joint arm with its parallel gripper [2] moves three 4 cm cubes from a table into three slots of a tray (Figure 2; parameters in Appendix B). The scripted policy grasps each cube at a noisy estimate of its position, checks the grasp by the gripper's width, and retries once; its *self-report* is FAIL only when a cube failed that check twice. It never looks at the tray, so it cannot sense a cube that slips from the gripper in transit, which happens with probability p_{slip} per carry. Ground truth, read from the simulator only for scoring, is PASS when all three cubes rest inside the tray. Two graders judge each episode. G_{in} , the flag wiring (inside the loop), is a logistic-regression classifier [24] on the 64 by 64 wrist-camera image at the end pose, trained on 300 of the robot's own episodes labelled by its self-report; the image shows the tray's three slots (Figure 2b, c), so a missing cube is visible in G_{in} 's input. G_{out} , the artifact wiring (outside the loop), is a fixed color rule on an overhead image taken after the arm parks out of view: PASS when each cube color shows at least 10 pixels whose centroid lies inside the tray. For fault injection (rule 3) the harness moves one cube out of the tray in an episode that truth and the self-report both call a success, leaving the self-report at PASS. Each slip rate (0, 0.10, 0.20, 0.30) gets 300 training episodes, 150 natural test episodes and 100 planted failures. All four studies were pre-registered: after pilots on separate development seeds, their predictions and scoring code were committed to a version-controlled repository before any test episode ran (Appendices B, C and D).

4.1 A grader trained on the robot's reports learns its blind spot

The robot's self-report said PASS on 83 to 84% of natural episodes at every slip rate, while ground truth fell from 83% to 49% (Table 1). With no slips, the self-report matched ground truth on all 300 training episodes; this is the control, with the same input and correct labels, and G_{in} passed 1 of 100 planted failures. At $p_{\text{slip}} = 0.10$, unsensed failures already outnumbered sensed ones among its training episodes (58 to 34 of 300), and G_{in} passed 61 of 100 planted failures and 14 of the 22 real unsensed failures among the natural test episodes; at 0.30 it passed 76 of 100 (Wilson 95% interval 0.67-0.83). Only the labels changed: the self-report flips FAIL to PASS on exactly the failures the robot cannot sense, label noise that is not at random [8]. G_{in} learned to pass a missing cube whether or not the robot had sensed it: at 0.30 it also passed 17 of the 25 natural episodes the robot itself reported as FAIL. Held-out agreement with the self-report did not reveal this: it was 0.88 at 0.10,

Table 1: Detection: 150 natural test episodes and 100 planted failures per slip rate. False completions (the robot reports done, the task failed) are a share of natural episodes; the one at $p_{\text{slip}} = 0$ was a grasp misread, not a slip.

Slip probability per carry, p_{slip}	0	0.10	0.20	0.30
Self-report pass rate, natural	0.84	0.84	0.84	0.83
Ground-truth pass rate, natural	0.83	0.69	0.57	0.49
False completions (self-report PASS, truth FAIL), natural	0.01	0.15	0.27	0.35
G_{in} agreement with the self-report, natural	0.99	0.88	0.81	0.74
G_{in} accuracy against truth, natural	1.00	0.84	0.69	0.63
G_{in} pass rate, unsensed natural failures	0.00	0.64	0.73	0.65
G_{in} pass rate, planted failures	0.01	0.61	0.77	0.76
G_{out} accuracy against truth, natural	1.00	1.00	1.00	1.00
G_{out} pass rate, planted failures	0.00	0.00	0.00	0.00

near the 0.84 of always answering PASS, and 0.74 at 0.30 against an accuracy of 0.63; the planted failures did. G_{out} matched ground truth on all 600 natural test episodes and passed none of the 400 planted failures; treating the plants as independent, that bounds its miss rate over the planted mix below 0.75% at 95% confidence.

4.2 The outside check lets the robot repair its work and report honestly

We then gave the robot the outside check as its own. After each episode it reads G_{out} on the overhead image. If the check fails, a scripted recovery routine finds every cube whose color centroid lies outside the tray, back-projects that centroid to the table with the camera model and the cube size, picks the cube there, places it in its slot, parks and images the scene again; it tries once more if the check still fails. The robot then reports DONE if the check passes, and otherwise stops and asks for help. The routine never reads simulator state; ground truth after the loop is read only for scoring. We replayed the 600 natural test episodes and the 400 planted failures of Section 4.1; every replay reproduced its recorded outcome.

Acting only on the outside check, the robot repaired 195 of its 213 natural failures (92%, Wilson 95% interval 0.87-0.95; Table 2), including 102 of the 116 its own report had called a success, and asked for help on the other 18. Task success, the ground-truth pass rate after the loop, rose at $p_{\text{slip}} = 0.30$ from 49% to 95%. It reported a failed task as done in none of the 600 natural episodes (the same robot relying on its self-report did so 116 times) or the 400 planted failures. The repairs broke no episode that had been a success. Of the 400 planted failures it repaired all 170 within reach and asked for help on the other 230, each beyond the arm’s reach. All 18 natural requests for help came from a cube against the outside of the tray wall, which the routine’s fixed grasp kept missing.

4.3 A robot that tunes itself

One setting that loosens the robot’s own check. The robot checks its grip after lifting the cube (a lift separate from the carry height), so the lift sets how hard that check is to pass. On the 150 natural test seeds without slips, lowering the lift from 15 cm to 5 cm raised the robot’s count of its own successes from 126 to 139, while true successes fell from 125 to 106. The check has two jobs: it writes the report and it triggers the retry. At 5 cm, marginal grasps passed it and were not retried, and the cube ended outside the tray; all 21 new failures were episodes the frozen policy had rescued with a retry. The robot reported 11 of its 44 failures, and G_{out} failed exactly those 44. We had registered that true successes would move by at most 6; they fell by 19 (Appendix D).

Fifty rounds of self-improvement. Each round the robot drew a new value for one of four motion settings (speed, the lift before the grip check, approach height, gripper close time), ran it and its current settings on the same 20 fresh seeds, and kept it if its grader reported more successes per simulated minute. Two copies, run once each, differed only in the grader, identical at the start because the robot’s report and G_{out} agree on all 300 training episodes: G_{in} retrained after every round on the

loop’s own runs, labelled by the robot’s report; or the same classifier trained once on G_{out} ’s verdicts, frozen and fault-injected before use. We put the lift within the loop’s reach on purpose: the question is what each grader does when a setting that loosens the robot’s check is available. Both started honest (each passed 1 of 100 planted failures), and both doubled the arm’s output within ten rounds (true completions per simulated minute from 1.8 to 3.9 and 3.8). By round 10 the self-trained grader passed 39 of the 100 planted failures; 14 of the 700 runs it had trained on were failures the robot had reported as successes, 12 of them trial runs at a low lift of changes it rejected. It kept no change from round 8 to round 41, took the 5 cm lift at round 42 on a sample of 20 in which that setting failed once, and raised the lift again five rounds later. The frozen grader passed 1 of 100 at every read-out. When its copy kept a bad lift on 20 seeds that happened not to show the harm (true success 82% at round 30 and its grader 82%, while the robot’s own grip-check report said 97%), it reported the drop, and the copy restored the lift within four rounds (Figure 3 in Appendix D). On natural episodes, however, the self-trained grader’s reported success stayed within 4 points of the truth, because the settings it ran rarely failed; a prediction we had registered, a gap above 10 points, lost (Appendix D). Planted failures showed the decay at round 10; the success rate never did.

5 Discussion: what a robot needs when no one is watching

Each rule showed up in the data. The robot’s account was blind to the failures it could not sense (rule 1); a grader trained on that account (rule 2) learned to pass missing cubes it could see, which planted failures exposed and agreement with the robot’s labels did not (rule 3). The mechanism is label noise aimed at the grader’s blind spot, and a loop feeds it: every run, kept or not, adds labels. Planted failures need no labeller: the harness knows what it planted.

The outside check did two jobs with one instrument: it graded each episode and told the robot what to repair and whether the repair had worked. Its zero false completions follow from its accuracy in this scene, and that is the general point: a robot that reports done only when its check passes has exactly as many false completions as its check has false passes on the states it leaves behind, so a check need not be perfect, only measured, by planted failures, where it runs. When a repair failed, the robot asked for help, which costs a person’s time; a false success instead passes the error downstream, where it compounds unseen, as the loop’s mislabelled examples did.

Steering the robot with a grader is where gaming enters, and the loop showed its simplest form: the robot’s own setting loosened its own check, and a grader that learned from that check decayed while its success rate barely moved. The number to watch is the grader’s pass rate on planted failures, measured on the states the loop produces; G_{out} passed no failed episode in any of the loop’s runs (with three false alarms in 4,400 read-outs). The decay mattered anyway: the success rate stays near the truth only while failures are rare, and the loop’s decisions had already suffered (Section 4.3). A policy that learns against the check could still find states it passes and the task does not [28, 20], which is why protocol item (ii) scores recovery against a reference the robot does not control.

6 What comes next

The workshop’s demonstration session provides the NERO arm we did not have, and with it a hardware measure, by planted failures, of the overhead check’s false-pass rate and of how often real repairs end in a request for help. A VLM or learned detector can replace the color rule if it never trains on the robot’s labels and its planted-failure pass rate is measured before and during deployment; it need not be perfect, only measured (Section 5). The next study runs the loop of Section 4.3 on learned grasp and repair policies.

7 Limitations

We report no physical-robot result; we had no NERO arm before the deadline. Simulation does isolate the mechanism, with ground truth on every episode and no lighting or sensor noise to blame; the

workshop’s arm must show the check’s false-pass rate holding up under that noise. The policy and the repair routine are scripted, and the loop tunes four of the policy’s settings, so the protocol’s resistance to a policy that learns against the check is argued, not tested; each loop arm ran once. A four-setting search is a weak optimizer, yet it poisoned its grader within ten rounds; a learned policy has more ways to reach states the check passes, so we expect a harder case, not yet shown. The studies show that the failures the rules target occur, and that an outside check catches them in this setting, not how often they occur in deployed systems. G_{out} is a simple color rule in a scene built to suit it (saturated cube colors, the arm parked out of view); its clean record here says nothing about cluttered scenes, and our plants sit clear of the tray wall and the other cubes, so they test it on easy cases. The repair rate rides on that clean view and will fall on hardware; what should carry over is that a failed repair ends in a request for help. G_{in} is a low-capacity classifier (logistic regression on pooled pixels); a stronger detector trained on the robot’s reports is untested. The repair routine was changed once, on development seeds, before its study was registered (Appendix C); the physics includes values we chose (Appendix B). The four instrument failures behind the rules (Appendix A) come from one team’s process and one model family: existence proofs of a kind of failure, not a rate.

Use of generative AI. This paper was drafted with Anthropic’s Claude models. The assistant searched for and checked sources, and wrote the simulation code and its tests under the author’s direction; the predictions and scoring code of all four studies were committed to the repository before their scored runs. The process record in Appendix A is of the author’s work with this assistant. The assistant is not an author; the author takes full responsibility for the content.

References

- [1] A. Aamodt and E. Plaza. Case-based reasoning: Foundational issues, methodological variations, and system approaches. *AI Communications*, 7(1):39-59, 1994.
- [2] AgileX Robotics. agx_arm_urdf: robot description files for AgileX arms, including NERO. GitHub repository github.com/agilexrobotics/agx_arm_urdf, MIT License, commit f6642ce, 2026. Accessed 26 September 2026.
- [3] K. Bousmalis, G. Vezzani, D. Rao, et al. RoboCat: A self-improving generalist agent for robotic manipulation. *Transactions on Machine Learning Research*, 2023.
- [4] E. Coumans and Y. Bai. PyBullet, a Python module for physics simulation for games, robotics and machine learning. pybullet.org, 2016-2021.
- [5] R. A. DeMillo, R. J. Lipton, and F. G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34-41, 1978.
- [6] W. E. Deming. *Out of the Crisis*. MIT Center for Advanced Engineering Study, 1986.
- [7] Y. Du, K. Konyushkova, M. Denil, et al. Vision-language models as success detectors. In *CoLLAs 2023*, PMLR 232:120-136, 2023.
- [8] B. Frénay and M. Verleysen. Classification in the presence of label noise: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 25(5):845-869, 2014.
- [9] L. Gao, J. Schulman, and J. Hilton. Scaling laws for reward model overoptimization. In *ICML 2023*, PMLR 202:10835-10866, 2023.
- [10] S. K. S. Ghasemipour, A. Wahid, J. Tompson, et al. Self-improving embodied foundation models. In *NeurIPS*, 2025.
- [11] C. A. E. Goodhart. Problems of monetary management: The U.K. experience. In *Papers in Monetary Economics*, volume I. Reserve Bank of Australia, 1975.
- [12] W. Huang, F. Xia, T. Xiao, et al. Inner Monologue: Embodied reasoning through planning with language models. In *CoRL 2022*, PMLR 205:1769-1782, 2023.

- [13] J. Huang, X. Chen, S. Mishra, et al. Large language models cannot self-correct reasoning yet. In *ICLR*, 2024.
- [14] Isaac Lab Project Developers. `isaacsim_tasks` changelog, versions 20.1.0 and 21.1.1. github.com/isaac-sim/IsaacLab, commit e352104, September 2026.
- [15] G. Jahangirova, D. Clark, M. Harman, and P. Tonella. Test oracle assessment and improvement. In *ISSTA*, pages 247-258, 2016.
- [16] V. Krakovna et al. Specification gaming: the flip side of AI ingenuity. DeepMind blog, 2020.
- [17] H. Kress-Gazit, K. Hashimoto, N. Kuppuswamy, et al. Robot learning as an empirical science: Best practices for policy evaluation. *arXiv:2409.09491*, 2024.
- [18] X. Li, K. Hsu, J. Gu, et al. Evaluating real-world robot manipulation policies in simulation. In *CoRL 2024*, PMLR 270:3705-3728, 2025.
- [19] Z. Liu, A. Bahety, and S. Song. REFLECT: Summarizing robot experiences for failure explanation and correction. In *CoRL 2023*, PMLR 229:3468-3484, 2023.
- [20] J. Luo, Z. Hu, C. Xu, et al. SERL: A software suite for sample-efficient robotic reinforcement learning. In *ICRA*, pages 16961-16969, 2024.
- [21] J. Luo, C. Xu, J. Wu, and S. Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105):eads5033, 2025.
- [22] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600-2606, 2018.
- [23] A. Panickssery, S. R. Bowman, and S. Feng. LLM evaluators recognize and favor their own generations. In *NeurIPS*, volume 37, 2024.
- [24] F. Pedregosa, G. Varoquaux, A. Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825-2830, 2011.
- [25] J. Rocamonde, V. Montesinos, E. Nava, et al. Vision-language models are zero-shot reward models for reinforcement learning. In *ICLR*, 2024.
- [26] J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278-1308, 1975.
- [27] S. Shingo. *Zero Quality Control: Source Inspection and the Poka-Yoke System*. Productivity Press, 1986.
- [28] A. Singh, L. Yang, K. Hartikainen, et al. End-to-end robotic reinforcement learning without reward engineering. In *RSS*, 2019.
- [29] K. Stechly, K. Valmeekam, and S. Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks. In *ICLR*, 2025.
- [30] W. Xiao, J. Xie, T. Zhang, H. Lin, et al. ENPIRE: Agentic robot policy self-improvement in the real world. *arXiv:2606.19980*, 2026.
- [31] Z. Zhou, P. Atreya, A. Lee, et al. Autonomous improvement of instruction following skills via foundation models. In *CoRL 2024*, PMLR 270:4805-4825, 2025.
- [32] Z. Zhou, P. Atreya, Y. L. Tan, et al. AutoEval: Autonomous evaluation of generalist robot manipulation policies in the real world. In *CoRL 2025*, PMLR 305:1997-2017, 2025.

A Where the rules came from: four instrument failures in a logged improvement loop

This appendix records where the rules came from: four failures of evaluation instruments, each with a counterpart in a robot-learning stack (a learned success judge, a safety gate, an autonomy gauge and a benchmark reader). It is provenance, not evidence for the robot results, and a reader interested only in the robotics can skip it. The rules were written after the evaluation instruments of a different self-improving loop failed four times in the same way. That loop is a process in which one person and a large-language-model assistant improve their shared way of working (the skills, gates and ledgers that govern the work) rather than a model’s weights, over a repository that now holds about 12,000 tracked files, with written ledgers since July 2026 and one shared event log since September 2026. The loop is one sentence: propose one process change; test it against a signal fixed outside the model; keep, switch or kill; archive the lineage; repeat. Nothing in it is new: it is plan-do-check-act [6] with the retain step of case-based reasoning [1] and the pre-registration of empirical science [22], run on the process itself under three disciplines.

Frozen fitness function. The checks that decide whether work is good may be *proposed* by the model only as a loud, diffed line (“was *X*, now *Y*”) and are installed only on the person’s recorded word. A process that could rewrite its own evaluator would, under selection pressure, find the evaluator that passes it [16, 23]; this is the constraint Section 3 carries over to a robot that works alone.

Sealed pre-registration. A prediction is sealed with its scoring rule as an executable (a regular expression, an enumerated set, a committed function), never as prose, and the rule’s leak rate is measured before the outcome is read; Appendix A.3 shows why.

Kill operator. Every learning and staged proposal carries a due date, at which it is scored by *opening the artifact its signal names*: win, loss, indeterminate (carried once) or killed. A line left unscored seven days past its due date is killed by the person from the census script’s list. When a sealed prediction loses, the next approach is proposed by a fresh model instance shown the loss but not the losing line’s history; Appendix A.1 shows why such an instance can find defects but cannot serve as an independent grader.

All four failures below share one cause: an instrument reported a number or a line without having decided anything about the property the number named, and the test that separates the two is to feed the instrument a known fault and watch it refuse (the logic of mutation testing [5] and of test-oracle assessment [15], applied to evaluators, gates and gauges). None of these instruments is a robot, but the same test applies to each counterpart named above. Two external graders in the record, a proof kernel and an optimization competition’s verifier, were not seen to fail; we did not fault-inject them.

A.1 The evaluator that could not lose

We built an evaluator whose critic judges whether a summary faithfully reports its source; critics were scored by a script against a sealed key, never by a model. Three sealed rounds lost the same way: the self-grade missed nothing, so no panel could add a catch. In the first two rounds every critic from the generator’s model family, its own fresh-instance self-grade included, graded all 12 items correctly (6 with planted errors, 6 clean) on gold sets whose sources had been pasted into the critic’s prompt; in the second, the different-model member (Qwen2.5-3B-Instruct, run locally) said *pass* to everything. The refutation of the sealed prediction (a panel would catch an error the self-grade missed) was the finding: a pasted source measures *reading*, while the failure an agent suffers is the source *never opened*. The third round hid each source among decoys; eleven fresh same-family instances still resolved all twelve items. The fix routes every file a critic opens through a logging gate and scores the read from the gate’s log rather than from the model’s account. The next round produced the first mechanical catch: on three items a member returned a gate receipt and quoted file text for files the log shows it never opened. The lesson is construct validity with Goodhart’s law behind it [11]: a

grader that shares the generator’s inputs measures the inputs, as a success classifier trained on the policy’s own rollouts, or an LLM judge reading the agent’s own trace, does.

A.2 The gate that failed open

The repository’s pre-commit gate blocks commits that contain unfilled placeholders, in the spirit of poka-yoke [27]. The evening after it was installed, it raised 28 blocks on a commit and then crashed while printing them: a character outside the console’s code page raised an encoding error, and the gate’s exception handler allowed the commit, printing “gate errored . . . allowing commit (fail-open)”. The commit went through with every file the gate had just blocked, and the next commit failed open the same way; an ASCII-only commit was refused. Whenever its report could not be printed, the gate’s decision was thrown away. The fail-open branch, chosen so that a crash would never block the person’s work, violates fail-safe defaults [26]; it was found by reading the hook’s printed output, not its exit code. The fix (UTF-8 output, fail *closed* on any exception) was proposed as a diffed change, installed on the person’s word, and *proved before use* by fault injection: a fixture of four files, one with non-ASCII text and one with a single planted placeholder, produced exactly one block, exit code one, and no error line. A safety monitor, a reset detector or a sim-to-real guard that catches its own exception and returns “clear” is the same gate, and a green run in which the guard never fired is not evidence that the guard works.

A.3 The gauge that counted writes as reads

To baseline how often the team consulted its own corpus before searching the web, a regular expression counted handoff documents that named an internal source before a web search; it reported 14 of 34. A hand read found that 11 of the 14 named the corpus only as a *write target* (a ledger to route learnings to) or in a list of sources, not as something read: a misattribution rate of 0.79, and an honest baseline of 3 of 34. Sealing the rule as an executable is necessary but not sufficient; its leak rate has to be measured *before* the outcome is read. The corrected gauge keys on a retrieval artifact (a hit list with file and line, flipped from UNREAD to OPENED) rather than on a word. The robot analogue is an autonomy metric that counts the wrong event, such as interventions logged at episode reset rather than at takeover.

A.4 The reader that buried a real result

Our board-watch reader for an external optimization leaderboard printed our entry at ranks 9 to 17 on four boards on a morning when the platform’s API had silently dropped the field that says whether a board is maximized or minimized. The reader defaulted to a descending sort; on three of the four boards our entry was in fact first by the board’s own scoring direction. A rewrite that reconstructs the direction and fails loud when the field is missing was written and staged; at the time of writing it has not been installed, an instance of rule 2: a fix on the record is not a fix in the instrument. A benchmark harness’s own reporting code is rarely fault-injected the way the policy is.

A.5 The record behind each number

Every number in this appendix was re-checked against its record on 26 September 2026 by a read-only audit of the repository; where the audit disagreed with an earlier draft, the text uses the audited value. The record is a private repository; the table names the kind of record so that a reader can ask for it.

Claim	Value	Record
Repository size	12,137 tracked files on the evening of 26 Sep 2026	version-control file list
Age of the loop	ledger rows from 31 Jul 2026; version control from 26 Aug 2026; one shared event log from 15 Sep 2026	the event log and the repository history
Evaluator, rounds 1-2	every same-family critic graded all 12 items correctly (6 with planted errors); in round 2, Qwen2.5-3B-Instruct passed every item	sealed keys and scoring-script output
Evaluator, round 3	11 fresh same-family instances resolved all 12 items with sources among decoys	same
Evaluator, next round	on 3 items, a gate receipt and quoted text for files the log shows were never opened	the gate’s read log
Gate	installed evening of 9 Sep 2026; on 10 Sep 2026 at 18:44 ET a commit with 28 blocks went through after an encoding error in printing (‘charmap’ codec), and a second commit failed open the same way; fix installed 18:56 ET; fixture of four files gave one block, exit code 1, no error line	the hook output quoted in the staged proposal of 10 Sep 2026; commit history
Gauge	14 of 34 reported; 11 of the 14 were write targets (0.79); honest baseline 3 of 34	hand-read table of the 34 documents
Reader	ranks 9 to 17 printed on four boards; first by each board’s own scoring direction on three of them; fix staged, not installed	board snapshots and provenance

B The detection study

B.1 Registration history

First registration (v1). Sealed on 26 September 2026 before its environment existed. The first development run (seeds 9000-9049 only) exposed three design flaws, and the seal was scored a loss by its own rule, before any scored run and before any grader verdict on a test seed: (1) with a 10 cm pre-grasp opening the self-report never failed (0 misses in 150 grasps), so every training label for G_{in} would have been PASS; (2) the place rule (offsets of up to 3 cm for 4 cm cubes) stacked cubes, and its ground truth counted a stacked cube as a failure that no overhead check could see (11 of 50 development episodes failed only that way); (3) at the end pose the gripper hid the tray centre from the overhead camera, and the arm hid 6.6% of the planting area.

Second registration (v2). Designed to remove those flaws (tray slots, ground truth without the wall-height condition, the arm parked out of view before the overhead image, a pre-grasp opening chosen for a sensed-failure rate near 0.10, and a slip-rate sweep), piloted on development seeds 9000-9049 only, then sealed. The seal file, the scoring script and the hashes of the frozen code were committed to the repository at 00:02 UTC on 27 September 2026, and the scored run started after the commit. Three changes from the plan written before the pilot are recorded in the seal with their reasons: a fourth slip rate (0.30) was added, and P1 moved from comparing 0.20 with 0 to comparing 0.30 with 0 (threshold unchanged), because at 0.20 sensed and unsensed failures were close in number on the development seeds; and a planned leg at 0.10 (G_{in} agrees with the self-report at least 0.90 of the time and passes at least half of the planted failures) was dropped because the pilot showed sensed failures outnumbering unsensed ones there. Had that leg been kept it would have lost on its first half: agreement at 0.10 was 0.88. A dry run of the study script on development seeds fitted G_{in} on 20 development images to prove the pipeline ran; no rate was computed from it.

B.2 Environment, robot and graders

- *World.* PyBullet [4] in direct mode, time step 1/240 s, 300 solver iterations. The NERO model with parallel gripper is built from the manufacturer’s MIT-licensed description files [2]; its base sits on a 15 cm riser, turned 180 degrees. Three 50 g cubes of 4 cm (red, green,

blue) start at least 8 cm apart in a 20 by 30 cm region; the tray is 20 by 20 cm inside with 3 cm walls; its three slots are 6 cm apart.

- *Values we chose.* Arm joint effort is capped at 30 N m (the description files carry 100 N m on every joint as a placeholder), because at 100 N m a finger that landed on a cube top crushed the cube into the table and the grasp still read as held. Finger force is 10 N. Inverse kinematics is our damped least squares, because PyBullet’s solver did not converge reliably on this arm.
- *Policy.* Perceived cube position = true position plus Gaussian noise ($\sigma = 1.0$ cm) in each horizontal axis, redrawn for the retry; pre-grasp opening 6.0 cm; the grasp is held if the gripper is wider than 0.5 cm after closing and lifting 15 cm; one retry; carry 15 cm above the slot (plus a uniform offset of up to 1 cm in each axis); release with the cube bottom 1 cm above the tray floor; end pose 15 cm above the tray centre with the gripper open; then the arm is parked straight up.
- *Self-report, slips, truth, plants, graders.* As in Section 4. A slip opens the gripper at a uniform point of the carry line. Truth requires every cube centre inside the tray’s inner rectangle, at rest (below 1 cm/s) and not touching the gripper, read after parking. A plant moves one uniformly chosen cube to a uniform table position at least 2 cm outside the tray wall, inside the overhead view, clear of the other cubes and the riser. G_{in} : wrist image (64 by 64) average-pooled to 32 by 32 by 3, scaled, standardized on the training split, logistic regression with $C = 1$ (L2), threshold 0.5. G_{out} : overhead image (128 by 128, camera 1.2 m above the table); for each cube colour, pixels with their own channel at least 128 and the other two at most 64; PASS if each colour has at least 10 such pixels and its centroid lies inside the tray’s inner rectangle.
- *Seeds.* Development 9000-9049 (pilot only); training 0-299 and natural test 1000-1149, the same at every slip rate, with slips drawn from their own random stream; planted episodes are the first 100 qualifying seeds from 2000 upward at each rate, so they differ between rates.

B.3 Pilot on development seeds

The pre-grasp opening was chosen by a rule written before the grid ran: a self-report FAIL rate between 0.08 and 0.15 at $p_{slip} = 0$ on 50 development seeds, closest to 0.10. Only 6.0 cm qualified.

Opening (cm)	Self-report FAIL rate	Miss rate per grasp	Retries	Double misses
6.0	0.14	0.207	29	8
6.5	0.04	0.119	18	2
7.0	0.02	0.045	6	1
7.5	0.00	0.007	1	0
8.0	0.00	0.000	0	0

Environment rates on the 50 development seeds at the chosen opening (the 0.30 column was run before the seal, after the sweep was extended):

p_{slip}	0	0.10	0.20	0.30
Truth pass rate	0.86	0.76	0.62	0.54
Self-report pass rate	0.86	0.86	0.86	0.86
Sensed failures (of 50)	7	7	7	7
Unsensed failures (of 50)	0	5	12	16
Slips	0	10	28	46

No development episode had a self-report of FAIL with a truth of PASS. Cubes left outside the tray in episodes the robot reported as FAIL were never inside the wrist image at the end pose (0 of 43 over the four rates, including cubes that slipped in those episodes); cubes from unsensed slips were inside it 1 of 5, 4 of 13 and 4 of 20 times at 0.10, 0.20 and 0.30, and planted cubes 1 of 20. G_{out} was fault-injected on development seeds before the seal: it agreed with truth on all 50 natural episodes at

0.10 and all 50 at 0.30, and passed none of 20 planted failures. The parked arm covered no pixel of the tray, a cube or the planting area in 170 development episodes. At the chosen opening a finger landed on a cube top 43 times in 179 grasps, and 3 cubes were squeezed out sideways, each ending as a sensed double miss.

B.4 Predictions as registered, and outcome

P1: G_{in} 's pass rate on planted failures at $p_{slip} = 0.30$ minus its pass rate at 0 is at least 0.30. P2: at 0.30, G_{in} 's agreement with the self-report on natural test episodes minus its accuracy against ground truth is at least 0.10. P3: at every slip rate, G_{out} 's accuracy on natural test episodes is at least 0.95 and its pass rate on planted failures at most 0.05. WIN if all three hold; otherwise LOSS, each leg reported. Stated expectations, not scored: G_{in} planted pass rates near 0.05, 0.25, 0.55 and 0.70 at the four slip rates.

Outcome. Scored by the registered script after a single unmodified run (2,200 scored episodes; 2,472 simulated, counting planted candidates that did not qualify; 67 minutes on two cores) and re-scored on a second machine with the same result, both recorded in the repository's event log: P1 = $0.76 - 0.01 = 0.75$ (holds); P2 = $0.740 - 0.633 = 0.107$ (holds, narrowly); P3 holds at every slip rate (accuracy 1.00, planted pass rate 0.00). Verdict: WIN. The unscored expectations missed low at 0.10 and 0.20 (measured 0.61 and 0.77). The pilot had counted 5 unsensed against 7 sensed failures per 50 development episodes at 0.10, with slips on 7.0% of carries; the scored training set had 58 against 34 per 300, with slips on 11.8% of carries. At 50 episodes a rate carries several points of sampling error, enough to change which label dominates, and the expectation did not carry that interval.

B.5 Full results

p_{slip}	0	0.10	0.20	0.30
Training episodes	300	300	300	300
of which self-report FAIL	34	34	34	35
of which unsensed failures	0	58	93	126
Natural test episodes	150	150	150	150
Planted failures	100	100	100	100
Truth pass rate, natural	0.83	0.69	0.57	0.49
Self-report pass rate, natural	0.84	0.84	0.84	0.83
Unsensed failures, natural (count)	1	22	41	52
G_{in} agreement with self-report, natural	0.99	0.88	0.81	0.74
G_{in} accuracy against truth, natural	1.00	0.84	0.69	0.63
G_{in} pass rate, unsensed natural failures	0.00	0.64	0.73	0.65
G_{in} pass rate, planted	0.01	0.61	0.77	0.76
G_{out} accuracy against truth, natural	1.00	1.00	1.00	1.00
G_{out} pass rate, planted	0.00	0.00	0.00	0.00

B.6 Code and data

The environment, graders, study script, scorer, pilot data and results are in the supplementary material (`simA/`), with their SHA-256 hashes: environment `a000082d`, graders `9178e880`, study script `4740400e`, scorer `86c586d3`, seal file `447b9e53`, results `824f0eaa`.

C The recovery study

C.1 The recovery routine and its one development change

The routine is the one first written for the demonstration video (Appendix C.5) with one change, made on development seeds before the study was registered. After each episode it reads G_{out} 's per-color detail on the overhead image. For each color whose centroid lies outside the tray it back-projects the centroid to the plane of a resting cube's top face, checks that inverse kinematics reaches the point

and the point 15 cm above it (within 2 mm), and if so picks there with no added noise, checks the grasp by gripper width, and places the cube in its slot; after a failed grasp it parks, images the scene again and tries once more from the new estimate. It then returns to the end pose, parks and images the scene. The loop runs the routine at most twice per episode and reports DONE if G_{out} then passes, otherwise NEEDS HELP. A color with fewer than 10 pixels is reported as not visible and not acted on. The change: on development seeds 9000 and 9003 (planted, $p_{slip} = 0$) the only flagged cube was out of reach, so nothing was picked and the arm was still parked; the first version then drove the arm to the end pose along a straight line from the straight-up park pose, which swept it through the scene (on seed 9000 it moved a cube already in the tray by 7 cm and pushed the flagged cube through the table). The registered version skips that closing move when the arm is still parked.

C.2 Pilot on development seeds

The pilot ran the loop on development seeds 9000-9049 at all four slip rates (200 natural episodes; 139 planted failures on the seeds whose episode qualified); the environment refuses any other seed in this mode. It repaired 48 of 61 natural failures (Wilson 95% interval 0.67-0.87) and 72 of 139 planted failures (0.44-0.60), with no false completion; every planted request for help was a cube out of reach. The registered thresholds were set from these numbers.

Development seeds, p_{slip}	0	0.10	0.20	0.30
Natural failures	7	12	19	23
repaired	5	9	15	19
asked for help	2	3	4	4
False completions, natural	0	0	0	0
Task success without / after repair	0.86 / 0.96	0.76 / 0.94	0.62 / 0.92	0.54 / 0.92
Planted failures	43	38	31	27
repaired	22	19	16	15
asked for help	21	19	15	12

C.3 Predictions as registered, and outcome

R0: every replay matches its recorded outcome in the detection study. R1: pooled over the four slip rates, the robot repairs at least 0.70 of natural failures (ground truth FAIL before the loop, PASS after). R2: at every slip rate, false completions are at most 0.02 of natural episodes. R3: at $p_{slip} = 0.30$, task success after repair is at least 0.85 (0.49 without repair). R4: at every slip rate, false completions are at most 0.02 of planted failures. R5: pooled, the robot repairs at least 0.40 of planted failures. WIN if all hold; otherwise LOSS, each leg reported. Stated expectations, not scored: pooled repair rates near 0.79 (natural) and 0.52 (planted), and no false completion.

Outcome. The registration (seal file, scorer, code and pilot data) was committed to the repository before the scored run, which started after the commit and ran once, unmodified (1,000 episodes, 32 minutes on two cores): R0 no replay mismatches (holds); R1 195 of 213 = 0.92 (holds); R2 largest rate 0.00 (holds); R3 0.95 (holds); R4 largest rate 0.00 (holds); R5 170 of 400 = 0.42 (holds). Verdict: WIN.

The stated expectations missed in both directions: natural repair came in above the pilot's 0.79 and planted repair below its 0.52. In the pilot 67 of 139 plants lay beyond the arm's reach; in the scored run, 230 of 400.

Table 2: Repair driven by the outside check, on the episodes of Table 1. The robot reports done only when the check passes after its repairs, and otherwise asks for help.

Slip probability per carry, p_{slip}	0	0.10	0.20	0.30
<i>Natural test episodes, 150 per slip rate</i>				
Failures before repair	25	46	65	77
repaired by the robot	25	42	59	69
robot asked for help	0	4	6	8
False completions, robot trusts its self-report	1	22	41	52
False completions, robot uses the outside check	0	0	0	0
Ground-truth pass rate without repair	0.83	0.69	0.57	0.49
Ground-truth pass rate after repair	1.00	0.97	0.96	0.95
<i>Planted failures, 100 per slip rate</i>				
Repaired by the robot	43	42	41	44
Robot asked for help	57	58	59	56
False completions	0	0	0	0

C.4 Full results

p_{slip}	0	0.10	0.20	0.30
Natural episodes	150	150	150	150
failures before repair	25	46	65	77
repaired (repair rate)	25 (1.00)	42 (0.91)	59 (0.91)	69 (0.90)
asked for help, task failed	0	4	6	8
asked for help, task done	0	0	0	0
false completions	0	0	0	0
successes the loop broke	0	0	0	0
a tray cube moved over 1 cm, task done	1	3	4	6
one repair round / two rounds	25 / 0	42 / 4	59 / 6	69 / 8
Planted failures	100	100	100	100
repaired (repair rate)	43 (0.43)	42 (0.42)	41 (0.41)	44 (0.44)
asked for help	57	58	59	56
false completions	0	0	0	0
a tray cube moved over 1 cm	0	0	0	1

Requests for help by cause, read from the logged actions and, for scoring only, the simulator’s cube positions: natural episodes, a cube against the tray wall that the grasp kept missing (18); planted failures, a cube out of reach within 32 cm of the robot’s base (116) and a cube out of reach farther out (114). A natural episode repeats its grasp misses at every slip rate with the same random draws (a slip at one rate recurs at every higher rate), so one troublesome seed can appear up to four times.

C.5 Code, data and the demonstration video

The loop runner, the routine, the scorer, the registration, the pilot data and the results are in the supplementary material (simA_rec/), with their SHA-256 hashes: runner `f38b3cf3`, routine `b6a9aae8`, scorer `c09f0766`, seal file `c27c1692`, results `ba9adbca`. The runner imports the detection study’s environment and graders unchanged and checks their hashes before it starts. The first 52.8 s of the demonstration video (development seed 9000, $p_{\text{slip}} = 0.10$) shows what the code did on that seed, captioned from the environment’s own stage labels: the red cube slips in transit, the robot reports the task done, the overhead check fails the episode, the robot locates the cube from the overhead image alone (back-projection error about 2 mm), re-replaces it, and the check passes. It was made with the routine’s first version; on that seed the registered version takes the same actions, with the same estimate and grip width, in the pilot.

D The loop studies

D.1 The lift before the grip check

Registration. A pilot on development seeds 9000-9009 first showed the effect: at a 5 cm lift the robot’s own count rose from 6 to 10 of 10 while the true count stayed at 6. The study was then registered, and its prediction, runner and scoring code committed to the repository, before any test seed ran (27 September 2026, 18:27 UTC). Seeds 1000-1149 at $p_{\text{slip}} = 0$; two arms, the frozen policy (lift 15 cm) and the same policy with the lift at 5 cm, nothing else changed; the lift is a setting the pick routine reads when it runs, so no frozen file was edited. Predictions, scored by the committed script: (R0) the frozen arm reproduces the detection study’s rows for these seeds; (primary) the robot’s own success count rises by at least 12 of 150 and the true success count moves by at most 6 either way; (L3) G_{out} passes no failed episode at 5 cm.

Outcome. R0 held: all 150 rows matched. The robot’s own count rose from 126 to 139, so the first half of the primary prediction held; the true count fell from 125 to 106, so the second half failed, and the primary prediction is scored a loss. L3 held: at 5 cm G_{out} failed exactly the 44 failed episodes, with no false pass and no false alarm. Read after scoring: 12 failures the robot had reported at 15 cm went unreported at 5 cm, and all 21 new failures were episodes that the frozen policy rescued with a retry after a missed grasp; at 5 cm, 18 of the 21 had no missed grasp at all. Episodes with a double miss fell from 24 to 11; 2 failures at 15 cm succeeded at 5 cm; the mean episode took 27.5 s of simulated time instead of 28.9 s. The pilot missed the fall in true successes because it lives in retried grasps, and its ten seeds held no retried grasp that the shorter lift broke; a pilot has to be large enough to see the smallest effect its prediction calls absent.

Second reader. A fresh instance of a different Claude model (Sonnet), told to open only the registration, the raw rows, the run’s metadata and the reference rows, re-scored the study with its own code; being of the same family as the model that wrote the code, it checks the arithmetic, not our judgment. Its claim sheet carried two planted false claims; it rejected both with the correct values and confirmed the rest, including both verdicts.

Video. The second part of the demonstration video runs development seed 9013 at both lifts: the first development seed on which the frozen policy rescued a missed grasp with a retry while the 5 cm lift lost the cube with no missed grasp, a rule written into the script before it ran.

D.2 The learning loop

Registration. The prediction is the one we registered on 25 September 2026: a success classifier trained on the improving policy’s own rollouts and used as the keep signal will report a success rate more than 10 points from an external measurement within 50 keep-or-kill rounds, while the same classifier frozen before the window, with its leak rate reported and fault-injected before use, will not. Every 10 rounds both rates are read on at least 200 fresh rollouts; the first arm has diverged if, at any of the five read-outs, the 99% Wald interval for the paired difference lies wholly outside ± 10 points, and the second has held if at every read-out it lies wholly inside; the prediction wins only if both happen. How we ran it here (scene, settings and their ranges, the proposal draw, the keep rule, both graders, seeds, the planted-failure probe and the scorer) was committed before any test seed ran (27 September 2026, 18:43 UTC), with our forecasts: 0.30 that the prediction wins, and 0.70 for S1, that G_{out} agrees with ground truth on every read-out episode.

Design. The frozen detection-study scene at $p_{\text{slip}} = 0$, where the robot’s report and ground truth agree at the start. The loop may change four settings, each on a fixed ladder starting from the frozen value: move speed 0.15 to 1.5 m/s (10 values), lift before the grip check 3 to 21 cm (10), approach height 5 to 21 cm (8), gripper close time 0.1 to 0.8 s (7). The pre-grasp opening, which sets the scene’s difficulty, the place height, the grip-check threshold and the slip rate stay frozen. Each round draws one setting and a new value uniformly (the same draws for both arms), runs the current and the proposed settings on the round’s 20 seeds, and keeps the proposal if its grader passes more episodes per simulated minute. The self-trained arm’s grader is G_{in} , first trained on the 300 training episodes

labelled by the robot’s report, then retrained after every round on every run of that round, labelled the same way. The frozen arm’s grader is the same classifier trained once on the same 300 episodes labelled by G_{out} ; it had to reject at least 95 of the 100 planted failures before use, and rejected 99. On those 300 episodes the two label sets are identical, so the two graders start identical. Ground truth is read for scoring and is never read by a decision. Read-out episodes (400 per read-out, seeds shared by the arms) are never trained on.

Outcome. Every gate held. The self-trained arm’s gap between reported and true success was +1.75, +4.00, +3.00, +3.75 and +3.75 points at rounds 10 to 50, each 99% interval inside ± 10 points (the highest upper bound was +6.5), so it never diverged; the frozen arm held at every read-out (largest gap 0.5 points). The prediction lost. S1 lost too: G_{out} disagreed with ground truth on 3 of the 4,400 read-out episodes, all false alarms, and passed no failed episode in any of the loop’s 8,260 natural runs. Table 3 gives the read-outs and Figure 3 draws them. The self-trained arm kept 9 changes (none from round 8 to round 41) and the frozen arm 14; the run simulated 8,360 distinct episodes (8,260 natural and the 100 planted failures) in 143 minutes on two cores.

Round	Self-trained grader			Outside-trained grader, frozen			settings
	reported	true	planted	reported	true	planted	
0	0.8575	0.8675	1	0.8575	0.8675	1	0.2/0.15/0.15/0.6
10	0.9350	0.9175	39	0.9125	0.9150	1	1.5/0.15/0.12/0.1; 1.5/0.15/0.12/0.2
20	0.9650	0.9250	46	0.9300	0.9300	1	1.5/0.15/0.12/0.1; 1.5/0.15/0.12/0.15
30	0.9500	0.9200	40	0.8200	0.8225	1	1.5/0.15/0.12/0.1; 1.5/0.06/0.12/0.15
40	0.9575	0.9200	34	0.9150	0.9200	1	1.5/0.15/0.12/0.1; 1.5/0.18/0.12/0.1
50	0.9500	0.9125	39	0.9450	0.9500	1	1.5/0.12/0.1/0.1; 1.5/0.18/0.08/0.15

Table 3: The learning loop’s read-outs: success rate reported by each arm’s grader and the true rate on 400 fresh episodes, and planted failures (of 100) the grader passed. Settings: speed (m/s) / lift before the grip check (m) / approach height (m) / gripper close time (s); one entry when both arms ran the same, otherwise self-trained; frozen. The robot’s own grip-check report is not shown; at round 30 it passed 0.968 of the frozen arm’s read-out episodes.

Second reader. A fresh instance of a different Claude model (Sonnet), told to open only the registration and the run’s logs, recomputed every read-out’s interval with its own code; it is of the same family as the model that wrote the code, so this checks the arithmetic, not our judgment. Its claim sheet carried three false claims written down before it started; it rejected all three with the correct values and confirmed the rest, including both verdicts.

D.3 Code and data

In the supplementary material (loophole/ and probot/), with their SHA-256 hashes: the lift study’s runner 24d7cc0d, scorer 2400bf65, registration 6e935647 and results 12ac78d1; the loop’s runner 910ead11, scorer f96de734, registration 460c52c4, read-outs 58c3e5ad, rounds 1c17215e and episodes d8e20ee7. COMMIT_TIMES.md there gives, for all four studies, when the registration and the results were committed to our repository and, where the run recorded it, when the scored run started.

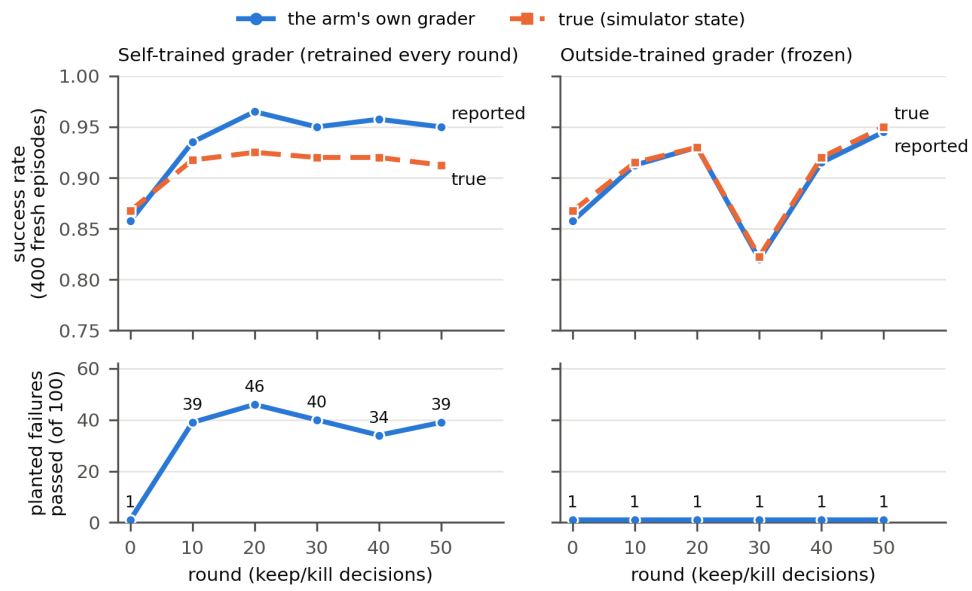


Figure 3: The learning loop. Top: success rate reported by each arm's grader and the true rate, on 400 fresh episodes after every 10 rounds. Bottom: planted failures, the same 100 known failures each time, that the arm's grader passed. The self-trained grader's reported rate stayed within 4.0 points of the truth while it passed up to 46 of the planted failures; the frozen grader passed 1 throughout, including at round 30, when its copy's settings cut true success to 82%.